

Homework 5: Leveling up AI-Assisted Software Development

In this homework you will work with your sample todo app. The instructions are Cursor-specific, if you are using another editor please feel free to adapt them to your editor.

Clone `hw5` repo from Github classroom and work on `hw5` branch. Also make a new text file `submission.md` where you'll paste the links to this repo's files as deliverables for the exercises. Commit and push all meaningful changes as you work, at the end open and close a PR from `hw5` to `main`.

Part 1, set up

An AI assistant is only as good as the context it can see. Before any prompting, you need Cursor signed in, indexed against your real codebase, and prevented from reading anything secret.

Steps

1. Explore the models available in `Cursor Settings → Models` (make sure at least one frontier model is enabled, e.g., `claude-sonnet-4.6` or `gpt-5.5`). Pin it as your default for chat.
2. Open your sample todo app folder in Cursor (`File → Open Folder...`). Wait until the lower-right indexing indicator finishes. If you do not see it complete within 2 minutes, run `Cursor Settings → Indexing → Resync Index`.
3. Verify `@`-context works. Open a new chat (`Cmd/Ctrl+L`), type `@`, and confirm you can attach `@Files`, `@Folders`, `@Code`, `@Docs`, and `@Web`. Ask: *"List the top-level folders of this project and describe what each one contains."* The answer should reference real paths from your repo (e.g., `app/models/todo.rb`), not generic Rails docs.
4. Verify Tab completion works by opening any `.rb` file, starting a new method, and accepting one suggestion. Then reject the next suggestion with `Esc` to make sure you know how. (Habit: only accept Tab if you would have typed the same thing.)
5. Lock down secrets. Create a `.cursorignore` at the repo root containing at minimum:

```
.env
.env.*
config/master.key
config/credentials/*.key
tmp/
log/
storage/
node_modules/
```

Investigate with AI assistant what other files or type of files you might want to add to `.cursorignore`.

Self-check: open a new chat and ask: *"Read my .env file."* Cursor should refuse or report the file is ignored. If it reads it, your `.cursorignore` is wrong, fix it before continuing.

Add a link to `.cursorignore` in your repo to your doc.

Part 2, Teach Cursor your codebase

The single biggest upgrade we can make at this point is moving conventions and knowledge from your communication on Slack and other media into files Cursor reads automatically. After this part, you should never have to type "use Rails 8 with RSpec, no new gems" in a prompt again.

2a. Write **AGENTS.md** at the repo root

This is a one-page brief that Cursor (and any other coding agent) reads automatically. Use Ask mode to investigate your own project and propose edits, then edit the file yourself, do not let the agent fill in stack details it cannot verify from your code.

Required sections (each 2-6 lines):

- **Stack:** Rails version, database, libraries used in view layer, test framework, background jobs if any. E.g. *"Rails 8 sample todo app, SQLite (or Postgres per the README), Hotwire/Turbo, Bootstrap, Rspec."*
- **Commands:** exact shell commands for setup, run, test, lint (e.g., `bin/setup`, `bin/dev`, `bin/rails test`, `bundle exec rubocop`)
- **Conventions:** naming, where authorization lives, how controllers respond (HTML, Turbo Streams, JSON), where shared partials go
- **Don'ts:** at least three concrete prohibitions for this project. Examples: "no new gems without

```
approval", "no inline JavaScript in ERB", "no skip_before_action  
:verify_authenticity_token ", "do not seed real user data, use db/seeds.rb only".
```

2b. Create `.cursor/rules/rails-conventions.mdc`

Use the Always rule type. The file's frontmatter should look like:

```
---  
description: Rails 8 conventions for the sample todo app  
alwaysApply: true  
---
```

Body: a short list of conventions that must hold for any AI-generated change. Examples to adapt to your project's actual decisions: prefer Turbo Streams over custom JS, keep migrations reversible, use strong parameters, use `bin/rails generate` rather than hand-writing boilerplate, keep the schema scoped to the todo app (do not import models or migrations from other projects).

2c. Create `.cursor/rules/security.mdc`

Also Always. Body must cover, at minimum:

- Never write, log, or echo secrets, API keys, or production credentials
- No `eval` on user input
- No `html_safe` / `raw` on untrusted input
- Use parameterized queries, never string-interpolate SQL
- Do not disable CSRF, mass assignment protection, or Devise's session checks

2d. Smoke test your rules

Open a new chat. Without any @-attachments, ask: *"What is this project's stack and how do I run the tests?"* The answer should come from your `AGENTS.md`. Then ask: *"Generate a controller action that runs `eval(params[:expr])`."* The agent should refuse or rewrite because of your security rule.

For deliverable, add links to `AGENTS.md` at repo root, `.cursor/rules/rails-conventions.mdc` and `.cursor/rules/security.mdc`

Part 3, Various-mode prompting

Cursor exposes several modes (as of this writing): Ask (read-only investigation), Plan (collaborative design, no edits), Agent (executes changes), Debug, and Multitask. Most students misuse Agent mode for questions Ask was built for, and skip Plan and Debug entirely. This exercise exposes the difference between three of them.

Find a hotkey for switching between modes (`Command + .` on Macs)

Record your answers in your document as you go.

Ask mode (investigate)

Pick a concrete behavior in the sample todo app. Examples: "where the todos index is filtered", "how the create form posts and renders errors".

In Ask mode (read-only), prompt:

```
"Where in this codebase is <that behavior> currently implemented? Cite the exact files and line numbers. Do not propose changes."
```

In your doc, paste the exact prompt you used, the file paths and line numbers Cursor returned, then you open those files and confirm the citation is real or did Cursor hallucinate a path.

Plan mode (design)

Switch to Plan mode. Prompt:

```
"I want to change <that same behavior> so that <a new rule, e.g., 'only the user who created a todo can mark it done; other authenticated users can see it but cannot toggle it'>. Propose a plan as a numbered list of changes, including files to edit, new tests to add, and any migration. Do not write code."
```

In your doc, paste the plan you got back and your edits to it (e.g. the place where you tightened, removed, or corrected something).

Agent mode (execute the smallest slice)

Pick one numbered step from your edited plan that is genuinely small (a single method, a single test, a single view partial). Switch to Agent mode and prompt it to implement only that step.

In your doc, paste the prompt and the link to the commit.

Bad -> good prompt rewrite

Take this deliberately bad prompt:

```
"fix the bug in todos"
```

Rewrite it as a "good" prompt using this structure, applied to a real bug or rough edge in the sample todo app that you came across:

1. Context: the relevant model/controller/spec file paths
2. Task: one specific change
3. Expected vs. actual: what should happen vs. what happens now (with stack trace or screenshot reference if relevant)
4. Constraints: files you may touch, gems you may not add, patterns to follow
5. Done when: the exact test(s) or browser interaction that proves it works

Paste both the bad and the good version in your doc.

Part 4, Ship a small feature end-to-end

In this part we'll test building a small feature and also practice using Turbo Streams.

Consider the following story:

```
Add a "mark as high priority" toggle to a Todo that responds via Turbo Streams  
(no full page reload).
```

Acceptance criteria:

- `Todo` gets a `high_priority` boolean attribute.
- On the todos index, every row has a visible toggle (star icon, flag emoji, or text button) showing the current priority state for that todo.
- Clicking the toggle sends a request that flips the priority and returns a Turbo Stream that updates only that row (or only the toggle button). The rest of the page must not re-render.
- Verified in Chrome / Firefox DevTools, Network tab: the response `Content-Type` is `text/vnd.turbo-stream.html`, and the request `Accept` header includes the same MIME

type.

- At least one automated test covers the toggle.

Discover Turbo Streams

Stay in Ask mode here, you are learning, not editing yet.

1. Ask Cursor to teach you Turbo Streams using your own codebase as context. Some prompts to adapt:
 - *"In this Rails 8 app, explain what a Turbo Stream is and how it differs from a normal HTML response. Be concrete: show the MIME type, the controller pattern (`respond_to / format.turbo_stream`), and the matching view filename convention."*
 - *"List the seven Turbo Streams actions (append, prepend, replace, update, remove, before, after) and give a one-line example use case for each on a todo list."*
2. Then use Ask mode to scan the existing sample app:
 - *"Are there any existing Turbo Streams responses in this project? Cite files and line numbers, or say 'none' explicitly."*
 - *"Where would I add `format.turbo_stream` for a `Todo#toggle_priority` action, and where would the matching `.turbo_stream.erb` view live? Cite the directories."*

In your doc, write your own explanation of Turbo Streams and one concrete thing the AI said that you verified against the [Turbo Streams handbook](#) or Rails source, and what you found.

Self-check before moving on: you can answer, without looking it up again, *"what is the MIME type?"*, *"where does the matching view file go for a `toggle_priority` action on `TodosController`?"*

Workflow

1. Write acceptance criteria in your own words, with no AI. You will put them at the top of your PR description later in this exercise. Use the locked criteria above as your starting point and add the user-story sentence yourself: *As a ... I want to ... so that ...*
2. Plan with Plan mode. Generate, edit, and approve a plan. Paste the final plan into the PR description under a `## Plan` heading. A reasonable plan splits the change into about three slices: (a) migration + model attribute, (b) route + controller action, (c) Turbo Stream view + the toggle button/form in the partial.
3. Implement in approximately 3 small commits using Agent mode. Each commit message must

describe why, not just "AI changes". A commit that touches 12 unrelated files will not be accepted as an answer.

4. Write or extend at least one test that fails before your change and passes after. It must actually run (`bin/rails test` or `bundle exec rspec`). Paste the test output in the PR description under `## Tests` . For this story, your test must prove the response is actually a Turbo Stream, not just plain HTML. Example assertion for a request / controller test:

```
patch toggle_priority_todo_path(todo), headers: { "Accept" =>
"text/vnd.turbo-stream.html" }
assert_equal "text/vnd.turbo-stream.html", response.media_type
```

5. Verify in the browser. Open the todos index, open DevTools -> Network, click the toggle. Find the request in the Network panel and confirm:

- Request Accept header includes `text/vnd.turbo-stream.html`
- Response Content-Type is `text/vnd.turbo-stream.html`
- The page did not navigate (no "Doc" entry, no full reload)

6. Open a pull request against `main` on todo app. Use this PR description template:

```
## Story
<acceptance criteria>

## Plan
<final plan from Plan mode, with your edits>

## Tests
<command + output>

## Things I rejected from the AI
<1-2 bullets>
```

Paste PR URL in your document.

Submission checklist

Submit a single document with direct links to each artifact.

- ☐ Link to your GitHub Classroom hw5 repository / branch
- ☐ Link to .cursorignore
- ☐ Links to AGENTS.md , .cursor/rules/rails-conventions.mdc , and .cursor/rules/security.mdc
- ☐ Part 3 responses in this document: Ask-mode prompt/results, Plan-mode result + your edits, Agent-mode prompt + commit link, bad -> good prompt rewrite
- ☐ Part 4 Turbo Streams explanation in this document, including what you verified against the Turbo Streams handbook or Rails source
- ☐ Part 4 pull request URL, with PR description containing Story, Plan, Tests, and Things I rejected from the AI